Blitzict: Structural Break Detection (0.879 recipe)

KU Applied AI Challenge qualification report · team **Blitzict** · 8 November 2026 · code: private repo `blitzict/ku-sbb-2026` · designated Kaggle submission **58102347** (0.88078)

In short. Each series has a known boundary point, and the model predicts the probability that its behaviour changes there. Our best recipe scores **0.8791 local out-of-fold ROC-AUC** and **0.88077 / 0.88078 on the Kaggle public leaderboard**. It works in three steps. First, it measures every series in several transformed views with eight groups of features. Then it trains 51 base detectors and a set of mechanism specialists on those features. Finally, a stack of small, regularised weight-space heads combines the detectors. We also kept simpler submissions: try1 (0.833), try2 (0.875) and **try3, a compact model with only 100 features + 36 prior columns per fold (0.87659 public)**.

1. Method

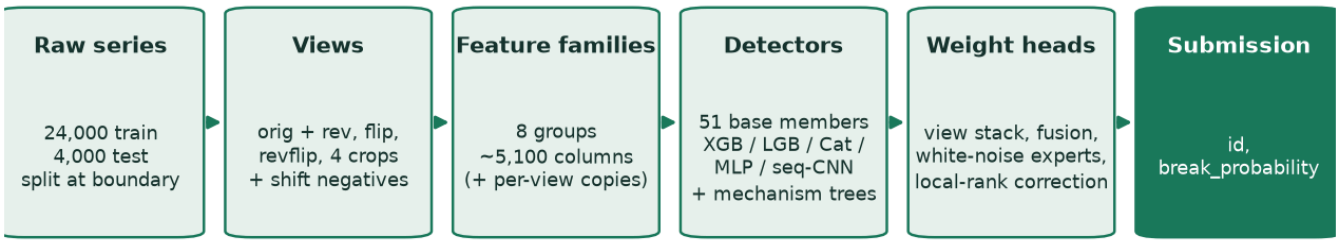


Figure 1. The 0.879 pipeline. Every arrow is a recorded step in `pipeline/full_cpu_stages.json` (473 steps). Each layer is trained only on out-of-fold outputs of the layer below.

Data and views. There are 24,000 labelled training series (12,000 with a break) and 4,000 test series. Each series is split into a "before" and an "after" part at its boundary, and almost every feature compares the two parts. To make the detectors robust, we build seven extra views of every series: time-reversed, sign-flipped, reversed and flipped, and four random crops around the boundary (Figure 2). Boundary-shift negatives and real half-segments add training rows. Augmented rows always stay in the same fold as their source series. At test time, the view models average their predictions over the views.

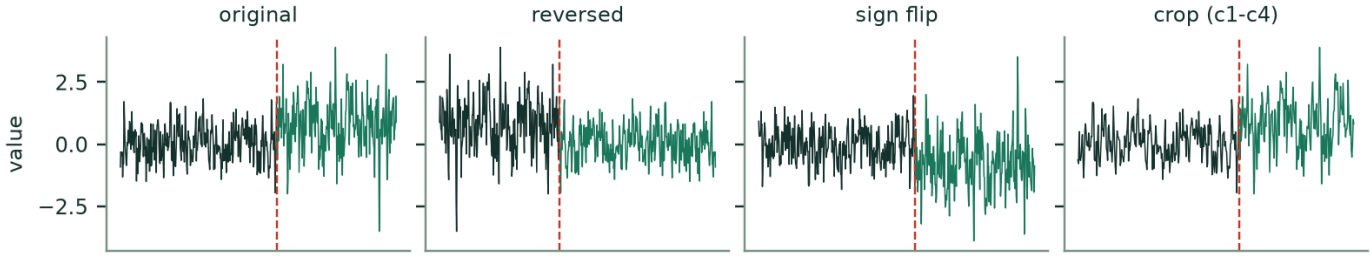


Figure 2. One illustrative (synthetic) series with a level and volatility break, and three of its views. The red dashed line is the boundary we score. The crops keep 60–90% of each side, with at least 80 points.

Feature families. The recipe uses about 40 feature tables. Table 1 groups them into eight families. The first four are classic two-sample statistics. The last four come from a mechanism view: they ask *how* the series could have changed (phase, memory, jumps, latent state, noise law) and score how likely that change is under each explanation. The full list is in `FEATURES.md`.

Family	What it measures (before vs after the boundary)	Columns
Base statistics	moments, quantiles, ranks, KS/AD/MMD tests, autocorrelation, spectra	977
Window grids	the same comparisons on two fixed window grids around the boundary	996
Increments & change tests	differenced series, likelihood ratios (raw + z), Bayes factors, self-normalised	433
Trend, volatility & spectrum fits	linear trend, GARCH, power spectrum	73
Periodic & phase	periodic components, harmonic fit, phase likelihood	91
Dynamics & memory	innovations, AR law, trend/curve, long memory, Markov copula, mode residual	288
Jumps, pulses & latent states	5 jump-law nulls, pulse/state filters, walk-vs-jump, boundary laws, latent level	1,246
Noise, tails & distribution laws	noise tests, bounds, mixtures, partition, tails, quantile alignment, native laws, volatility laws	983
Total distinct columns (before the per-view copies)		**5,087**

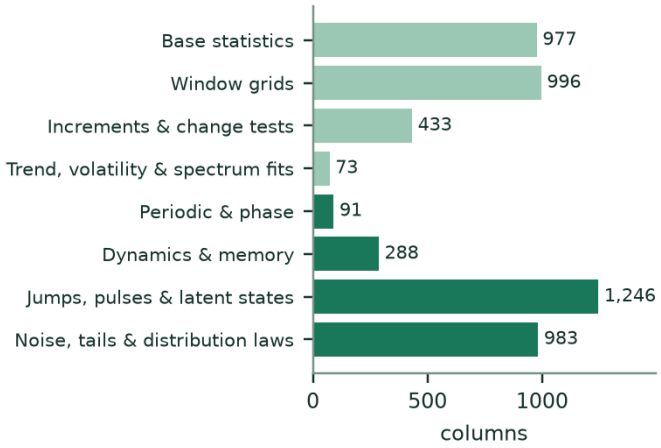


Figure 3. Size of each feature family. The mechanism families (dark) are the largest share, and they are what lifts the model from about 0.85 to 0.875+.

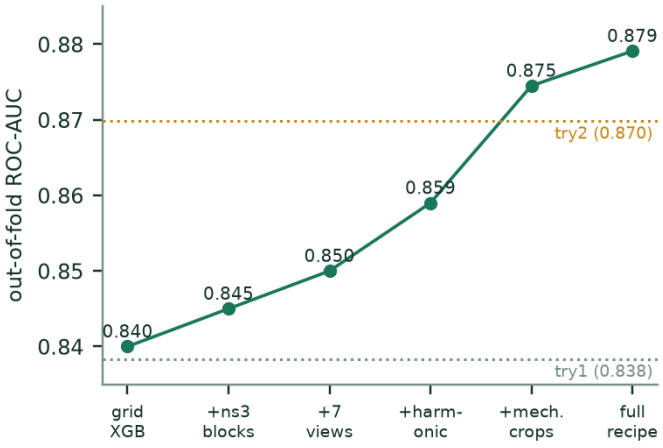


Figure 4. Historical out-of-fold AUC as parts of the recipe were added, compared with our two simpler submissions.

Detectors. These are the 51 base members:

- XGBoost, LightGBM and CatBoost models on selected statistical blocks of 170, 191, 206 or 2,247 columns (`train_cv.py`, 44 fits).
- View models trained on all 8 views, with test-time averaging (`train_var.py`).
- Multilayer perceptrons on a 2,171-column dense block, with rank-Gaussian inputs, 1024×3 hidden layers and dropout 0.4 (`nn_var.py`, `nn_xpu.py`).
- A dilated 1-D CNN on the raw padded series (`nn_seq.py`).
- Distilled trees.

The **mechanism specialists** are nested XGBoost models (`nested_tree/crop/rank.py`, `wnlaw_nested*.py`). They use the mechanism features plus noise-law priors, and each prior is estimated inside the training fold only. On its own, the crop specialist MECH3C12V reaches 0.874.

Weight-space heads. Instead of one big meta-model, we edit and recombine existing weights in small, regularised steps:

- Neural output-layer edits (`neural_head_edit.py`) and learned per-view weights.
- Expert fusion (ridge penalty 700) and mechanism, trend and extended readouts (penalties 40, 60 and 35).
- Specialists for series that look like white noise (rank boosts, crop laws, pair trees).
- The final readout, which mixes 75% of a three-source vote with 25% of the upstream branch.
- A **local-rank correction**, which compares each positive with 32 nearby negatives (seed 1729, logistic temperature 0.55, penalty 84) and is fitted with L-BFGS-B. Half of its coefficient vector is applied.

2. Validation

Base members use five stratified folds (seed 7, plus the recorded seeds for repeated members). The later readouts are refitted over five outer folds for each of six seeds (7, 29, 0, 123, 2024, 11041), with inner folds wherever a layer needs its own out-of-fold inputs. The headline AUC is the pooled seed-7 out-of-fold score over all 24,000 series. Learned preprocessing, such as rank references, quantile transforms and noise-law priors, is fitted on training rows only, and test labels are never used. **Limitation:** feature lists and settings were chosen while looking at the same training set, so the local score carries some selection optimism. It is a research validation score, not an untouched holdout.

3. Results

Submission	Local OOF	Kaggle
try1: v4, XGBoost + LightGBM, 977 base features	0.83835	0.83313
try2: one LightGBM, 3,625 statistical + mechanism features	0.86987	0.875
try3: full 0.879 ensemble	0.87911	0.88077
try3 + synthetic-data MLP correction (58102347)	0.87910	0.88078
try4 (selected): slimmed try3, top-100 features + fold-wise law prior, 2 LGB + 1 XGB, 22 min on 2 cores	0.87235	0.87583

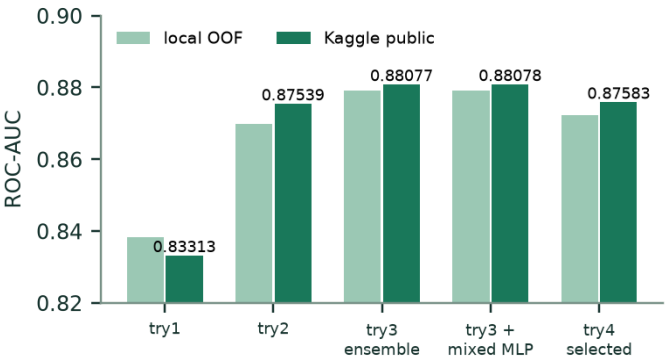


Figure 5. Local versus Kaggle public AUC for each submission.

The biggest single jump came from detecting **coherent oscillations**: a series can change its phase or amplitude while its mean and variance stay the same. Adding these detectors moved one ensemble from 0.853 to 0.867. Later gains came from the trend and pulse handling, the noise-law priors, crops that keep track of their source series, and the constrained residual heads. The hardest remaining

cases are breaks that look like white noise. Of 821 historically missed positives, the final ensemble recovers 88 at a 20% false-positive rate, compared with 42 for the earlier 0.871 model.

4. How we implemented and rerun it

The original models were trained on our own GPUs, but many intermediate models were saved only as predictions. To make the result reproducible, we rebuilt the whole recipe so that it runs from the four raw competition files:

- The frozen scripts are in `pipeline/legacy_cpu/` (172 files), and every step's arguments, inputs and outputs are recorded in the stage manifest.
- A resumable runner verifies the hashes of every input, source file and output, and **saves every fitted model**.
- `pipeline/train_gpu.py` runs independent steps in parallel and sends model fits through `gpu_launch.py`: XGBoost runs on CUDA (RTX 5070) and the torch nets run on the Intel Arc. The script files on disk stay unchanged, so their recorded hashes stay valid.
- Prediction loads the saved models and rebuilds the features for new series with the same code. For try1 this reproduced the submitted CSV exactly. For try2, all 3,625 features and every prediction matched exactly on 200 test series.

`python train_087910_cpu.py --data data --work cpu_run` (or `python -m pipeline.train_gpu ...`) trains the full recipe. `python -m pipeline.predict_new --models cpu_run --series test_series.parquet --csv submission.csv` scores new series. The designated CSV is written by `python try3/designated_088078/predict.py --data data --out submission.csv`, and `try2/predict_compact.py` and `try1/v4.py predict` do the same for the simpler models.

Selected model. Inference must run in 30 minutes on 2 CPU cores with 8 GB RAM. The full recipe (try3) needs hours, so we selected **try4**, its slimmed version: 136 columns per fold from 35 label-free tables, only the feature columns the models read, and 3 tree models. It keeps 99.4% of try3's public score and runs in 22 minutes with 2.6 GB RAM, measured on Kaggle pinned to 2 cores.

What we have (repository `blitzict/ku-sbb-2026`)

Item	Where	Status
try1: v4 models, train and predict code	`try1/`, `pipeline/features_v4.py`	Saved models; predicting from raw data reproduces the submitted CSV exactly
try2: 5 LightGBM fold models, train and predict code, notebook	`try2/`	Saved models; Kaggle 0.875; raw-series prediction matched exactly
try3: full 0.879 recipe (473 steps, 172 scripts)	`pipeline/`, `train_087910_cpu.py`, `try3/`	Retrained from raw files: local AUC 0.87886. Inference takes hours, beyond the 30 min / 2 core / 8 GB limit
try4 (selected): slimmed try3 (100 features + 36 prior columns per fold, 15 saved models)	`try4/`	Saved models; Kaggle 0.87583; scores the 4,000 raw test series in 22 min on 2 cores, 2.6 GB RAM
GPU runner (RTX 5070 + Intel Arc, parallel steps)	`pipeline/train_gpu.py`, `gpu_launch.py`	Done: all 473 steps
Saved-model inference for the full recipe	`pipeline/inference/`, `predict_new.py`, `inference.ipynb`	Not verified end to end; not needed for the selected try4
Feature catalogue and training notebook	`FEATURES.md`, `CPU_087910.ipynb`	Done
Designated 58102347 (0.88078): final layer, saved inputs, MLP fold models	`try3/designated_088078/`	`predict.py` rewrites the submitted CSV exactly (same SHA-256); the MLP part reruns from raw series. Upstream detectors replay saved outputs; their fresh retrain is in try3

5. Sources and AI use

Libraries: NumPy, SciPy, pandas, PyArrow, scikit-learn, XGBoost, LightGBM, CatBoost, PyTorch (CUDA / XPU), joblib and Optuna (development tuning only). Only the competition files were used: no outside data, no pretrained models and no test labels. The synthetic series used in some experiments were generated from training negatives. **AI assistance:** Claude Opus 5.5 (Claude Code) was used to implement ideas we had and created. It optimised the training for our two-GPU setup (RTX 5070 + Intel Arc), and later reconstructed the extracted features as a CPU-only run. We chose the ideas and the features, reviewed the results and decided what to submit.